L3 Patterns-in-Code Document — HELEP

> *~3 pages. Each pattern entry includes a **code citation** (`file:line`).*

Part A — Pre-implemented patterns

Identify and explain each pattern below as it appears in the starter code.

A.1 Choreographed Saga

- **Where:** `sos-service` publishes `sos.triggered` and `sos.cancelled` events in the REST handlers (`services/sos-service/app/main.py:78-110`). `dispatch-service` consumes those events and publishes `responder.assigned` and `safety.zone.entered` (`services/dispatch-service/app/main.py:62-93`). `notification-service` consumes those events and publishes `notification.sent` (`services/notification-service/app/main.py:52-71`).
- **Compensation step:** in `dispatch-service`, the cancel handler releases the assignment and marks it RELEASED, then publishes `responder.confirmed` (`services/dispatch-service/app/main.py:96-102`).
- **Rollback trigger:** the `sos.cancelled` event emitted by `sos-service` (`services/sos-service/app/main.py:102-110`).

A.2 Pub/Sub via Apache Kafka

- **Where:** producer/consumer helpers and manual commit live in `app/events.py` (`services/sos-service/app/events.py:105-139`).
- **Consumer group semantics:** auto-commit is disabled and commit happens **only after** handler success (`services/sos-service/app/events.py:127-139`), so failed handlers are retried (at-least-once).
- **Partition keying:** saga events are keyed by `incident_id` to preserve ordering; e.g., `sos.triggered` publish uses `key=iid` (`services/sos-service/app/main.py:85-96`) and `responder.assigned` uses the same key (`services/dispatch-service/app/main.py:77-81`).

A.3 Repository

- **Where:** database access is encapsulated in per-service repositories (e.g., `services/user-service/app/db.py:1-69`).

- **Why:** route handlers are thinner and data access stays consistent; direct SQL from handlers would duplicate logic and weaken invariants (e.g., unique constraints, transaction scope).

A.4 Strategy

- **Where:** responder matching strategies in `dispatch-service/app/matching.py` (`services/dispatch-service/app/matching.py:31-79`).
- **How to switch:** environment variable `MATCHER` selects the strategy (`services/dispatch-service/app/matching.py:73-78`).
- **Third strategy added:** `RoundRobinMatcher` cycles through responders (`services/dispatch-service/app/matching.py:57-70`).

A.5 Outbox-lite

- **Where:** SOS trigger handler inserts into SQLite, then publishes in the same async block (`services/sos-service/app/main.py:83-96`).
- **Why is this "lite"?** there is no durable outbox table or relay process; publish happens inline without transactional guarantees across DB and Kafka.

A.6 Circuit Breaker (stub → complete it)

- **Where:** `events.py` `class CircuitBreaker` (`services/sos-service/app/events.py:58-99`).
- **Task completed:** `allow()` now implements CLOSED → OPEN → HALF_OPEN and resets on success/failure (`services/sos-service/app/events.py:68-99`).
- **State transitions:** failures reach threshold → OPEN; after `reset_after_s`, a single probe is allowed (HALF_OPEN); success closes and resets counters, failure re-opens.

Part B — Patterns you added (minimum 2)

B.1 API Gateway

- **Where:** umbrella Ingress routes `/api/*` paths to services (`charts/helep/templates/ingress.yaml:1-30`).
- **Problem solved:** provides a single entrypoint and hides internal service topology.
- **Trade-off:** centralizes traffic and can become a bottleneck; path rewrites and auth must be managed consistently.

B.2 Autoscaling (Elasticity)

- **Where:** HPA scales deployments by CPU (`charts/helep/charts/user-service/templates/hpa.yaml:1-18`).
- **Problem solved:** handles load spikes without manual intervention.
- **Trade-off:** relies on metrics availability and introduces scaling lag; can oscillate under bursty traffic.

Part C — Anti-patterns avoided

Shared database across services is avoided: each service reads/writes its own SQLite file (e.g., `DB_PATH=/data/user.db`, `services/user-service/app/db.py:8`), preventing tight coupling and cross-service schema contention.

Submission

Submit as `patterns.pdf`. Keep code excerpts ≤ 10 lines.